Yuno — Understanding & Approach

> Submission document. Companion to the GitHub repo, recorded demo, and architecture / plan / setup docs. Repo: https://github.com/ashwanijha04/yuno-orchestrator

---

1. What the brief is actually asking

Beneath the surface (CRUD for agents, workflow templates, a Telegram bot), the brief is asking for **a real orchestration platform, not a hardcoded demo**:

- "Agents must run on a real runtime" → a generic executor, not a scripted skit
- "Execute real tools" → tool calls land as first-class steps with cost + latency
- "Communicate with each other to complete tasks autonomously" → both pipeline-style hand-offs and dynamic delegation
- "At least one agent reachable through an external messaging channel" → a real Telegram round-trip, not a webhook stub
- "Run fully local with a single setup command" → no cloud dependency, no fragile cleanup

The grading weights (40% working demo / 30% architecture & code quality / 20% UI/UX & configurability / 10% docs) confirm the priority: **a working end-to-end system, built well, that someone else can reproduce.**

The trap I tried to avoid: building one hardcoded workflow that demos beautifully but doesn't generalise. The platform test is whether **adding a new agent or workflow is a DB row, not new Python.**

---

2. Three load-bearing decisions

Every later decision in the codebase resolves to one of these three.

2.1 Workflows are graphs of agents; agents are graphs of reasoning steps

Two distinct concerns with two distinct execution graphs:

- **Outer graph** — workflow nodes (`agent` , `tool` , `condition` , `human` , `channel_out`) connected by edges that can carry DSL conditions (e.g. `iteration_count < 4`). Compiled per run from the workflow's JSONB graph.
- **Inner graph** — the ReAct reasoning loop *inside* each agent node: `prepare → llm → router → {tool | end}` . Same code for every agent.

Conflating these would be the most common failure: it produces "workflow builders that leak LLM concepts and reasoning loops that leak orchestration concepts." Keeping them separate means

the visual builder doesn't have to know about tool-use loops, and the agent executor doesn't have to know about graph topology. **See `backend/app/runtime/{outer,inner,engine}.py`.**

2.2 Agents are configuration; the runtime is code

An "agent" in the database is a row — name, role, system prompt, soul.md, persona, model binding, tool list, memory strategy, guardrails. The code that *runs* an agent is one generic executor.

This is the platform test: **if creating a new agent requires writing Python, the abstraction is wrong.** Yuno passes it. The seed script (`backend/scripts/seed.py`) creates 17 agents and 5 workflows entirely declaratively. The visual builder writes the same JSON the seed does.

2.3 Every side effect is a row first, then transport

LLM calls, tool invocations, inter-agent messages, channel events — they all write to Postgres **before** they hit Redis. Redis carries pub/sub events for the live UI; Postgres is truth. If Redis dies, in-flight runs fail but no data is lost. The live timeline, cost ledger, and replay capability all derive from the same tables — no separate logging path.

This is what makes the **run-detail timeline** trustworthy: it's reading the same rows the executor wrote synchronously. Not eventual-consistency telemetry.

---

3. Stack choices, justified

Choice	Why	What I rejected
Python for the runtime	LangGraph and the major LLM SDKs are Python-first; pgvector + extremis are native. Reasonable people would pick Node — this came down to library maturity.	Node (Vercel AI SDK was tempting but ecosystem for agent runtimes is younger)
LangGraph for the outer workflow graph	Workflows are genuinely graphs with conditional edges and feedback loops; LangGraph gives a real, inspectable `StateGraph`.	Hand-rolled FSM (would re-invent state checkpointing); CrewAI/AutoGen (too opinionated about agent topology)
Hand-written inner ReAct loop	LangGraph's prebuilt agent wouldn't give per-step control over the harness, persistence, or guardrails. Worth the ~120 lines.	LangGraph's `create_react_agent` (loses control over harness chokepoint)
FastAPI + Next.js + Postgres + Redis	Boring, well-understood, fits local-first.	Vercel (no path for long-running workers and WebSockets without a rearchitect)
Telegram via long-polling for the demo	No public URL needed; just works on a laptop. Webhook mode documented as the production path.	Webhook-only (would need ngrok/cloudflared every demo)
extremis for long-term memory	Embedded in the backend (no separate service), backed by Postgres+pgvector. Honest "what is this agent remembering across conversations" demo.	Vector DB SaaS (would break "fully local" requirement)
shadcn aesthetic discipline + a HUD identity layer	shadcn for dense surfaces (forms, tables, inspector). One bold identity layer (Stark-HUD: deep navy, holographic cyan, mono telemetry) for the cockpit + run timeline. Avoids the "generic AI dashboard" look while staying legible.	Pure shadcn (legible but characterless); pure custom design (didn't have the time budget)

The READMEs in `docs/architecture.md` and `docs/plan.md` defend each of these in detail.

---

4. Brief requirements ↔ where they live

Every line of the challenge PDF, mapped to running code + a timestamp in the recorded demo:

Brief requirement	Code	Demo timestamp
Create AI agents (personality, tools, schedules, memory, limits)	`backend/app/db/models.py:Agent` · `frontend/components/agent-form.tsx`	§1 cockpit · §8 list · **§8b 4:05 — full edit form**
Agent CRUD: name, role, system prompt, model, tools, channels	`backend/app/api/agents.py`	§8 + §8b
Agent configuration: schedules, memory, skills, interaction rules, guardrails	`Agent.{memory_policy, guardrails, harness}` JSONB · `app/memory/` · `app/guardrails/`	**§8b 4:05**
Connect into collaborative workflows	`backend/app/runtime/outer.py` (LangGraph)	§2 Cited Research · §3 PRD with Approval
Real runtime, execute real tools	`backend/app/runtime/engine.py` + `backend/app/tools/`	**§2 0:36 (TOOL node)** · §6 (http_request)
Communicate with each other to complete tasks	`tools/send_to_agent.py` (async, run-per-message + inbox) · `output_key → input_mapping` for pipelines	§3 (Pip → Mara → Brie) · §1b (Jarvis delegating)
≥1 agent on Telegram	`backend/app/channels/telegram.py` (long-poll + webhook) · channel_bindings table	§5 ACTIVE pill · **§6 0:00 real round-trip**
Web UI for managing everything visually	`frontend/app/{agents,workflows,runs,team,channels}/`	every section
Async agent-to-agent	`send_to_agent` enqueues a new run via the queue; parent waits on the child run row, not the LLM call	§3 (pause/resume) · §1b (Jarvis dispatching)
Message history persisted + visible	`messages` table → run-detail timeline reads from DB	**§6 3:14 — historical conversation reloaded**
Runtime actually executes logic (not a mockup)	LLM calls cost real money, tracked on `runs.total_cost_usd`	§1, §9 (gauges move after every section)
Visual workflow builder with conditions +	`frontend/components/workflow-builder/builder.tsx` (React Flow) · `runtime/dsl/` (Lark grammar)	**§3b 2:24 — Draft & Critique**

Brief requirement	Code	Demo timestamp
feedback loops		**loaded; `iteration_count < 4` condition called out**
≥2 pre-built templates	`backend/scripts/seed.py`	§2 (5 shown)
External channel integration	`channels/{base,telegram,slack,whatsapp,registry}.py` — adding one is a class + a registry line	§5 + §6
Live monitoring with real-time logs + inter-agent messages + cost tracking	`app/observability/events.py` → WebSocket → `frontend/components/live-run.tsx`	§2 timeline animates live
Working end-to-end demo with 2+ agents	—	§2 (3-agent) · §3 (4-step + human)

The audit is also in `demo/README.md` with the literal timestamp-to-requirement table.

---

5. What I cut, and why (judgment, not omission)

Cut	Reason	Status
WhatsApp + Slack channels as fully wired adapters	Telegram covers the brief requirement. Adding two more would have eaten budget.	Documented stubs in `app/channels/{slack,whatsapp}.py` proving the `Channel` abstraction — diff to add a real one is small, no orchestrator/agent/workflow changes.
Bedrock provider	Anthropic direct + OpenAI + Gemini = three live providers, covers the routing-and-fallback narrative.	Mentioned as future work. The `LLMProvider` protocol makes adding Bedrock a single class.
Eval framework UI (datasets / runs / scores)	The harness has the seam (`EvalRecorderInterceptor`, `LLMJudge` as itself a `HarnessedCall`), and there's an evaluate-button on each run that runs a judge inline. The full eval-runs UI was a stretch.	Inline evaluate (👍/👎 + judge) ships; bulk eval framework deferred.
`parallel` **node** (fan-out N children, join)	Schema admits it; executor returns "not implemented" with a clear message.	The differentiator most candidates won't have; honest about scope.
`transform` **+** `channel_in` **nodes**	Same — schema admits them, executor stubs them.	Stubs over fake implementations — I'd rather say "not built yet" than fake it.

The "what I cut" table is also in the README, called the "scope vs cut" table. **Shipping it visibly signals judgment.**

6. The risks I was explicit about and how I mitigated them

Risk	Mitigation
Demo recording is the highest grade weight (40%) but the most fragile thing	Built `demo/demo.mjs` (Playwright) + `narrate.mjs` (OpenAI TTS + ffmpeg mix). Demo is reproducible end-to-end with two commands; new takes take 6 minutes total.
"Run fully local with single command" is easy to break with hidden cloud dependencies	extremis runs **embedded** in the backend (Postgres-backed, no separate server). Tavily / Telegram / LLM keys are all *optional* — Yuno boots in `stub` mode (deterministic, no network) without any key.
Telegram is the most fragile external	Default to long-polling (no public URL needed). Webhook mode documented as the production path.
Cost cap should *demonstrably* protect runs, not just exist as a feature	`CostCapInterceptor` is in the harness *before* every LLM call; the demo's §4 trips it live with a `$0.0001` cap, shows the diagnostic on every step, $0 spent.
Test flakiness with shared pgvector memory	Honest disclosure in `docs/local-setup.md` § 5 troubleshooting — there's one pre-existing test isolation bug I caught and noted (`codename%` memories pollute pgvector across runs). One-line fix queued.

7. What I'd do next, with another week

1. **Move the cost-cap from graceful blocker to a policy** — current behavior is "block individual calls when cap would be exceeded"; could optionally be "fail the run hard with a red termination block" via a single flag. The graceful design is better for production but the harder failure is more visual.
2. **Wire `parallel` node** — fan-out N children + join. Biggest missing capability. Schema already admits it.
3. **The eval framework UI** — datasets, eval-runs, sparkline of pass-rate per template. The harness primitives all exist; the UI is the missing piece.
4. **Recording / replay end-to-end** — `LLM_MODE=record` and `LLM_MODE=replay` exist; the recording library and replay roundtrip test ship. The polish gap is the UI for managing recordings.
5. **Production deployment story** — DigitalOcean droplet path documented; would actually deploy + put a domain in front + add Basic Auth.

8. The five documents I'm submitting alongside this one

Doc	What it answers
`understanding.pdf` (this)	Why I built it this way
`architecture.pdf` (`docs/architecture.md`)	The system design, in detail
`plan.pdf` (`docs/plan.md`)	The execution plan I followed
`local-setup.pdf` (`docs/local-setup.md`)	How to run it on your laptop
`demo-coverage.pdf` (`demo/README.md`)	Timestamp-by-requirement map for the recorded demo

The recorded demo itself is at `demo/yuno-demo.mp4` in the repo (4:42, captioned + narrated).

Built with care for the Yuno AI Engineer challenge. Honest about what's shipped and what's stubbed. Repo at https://github.com/ashwanijha04/yuno-orchestrator.